

Introduction to DOM

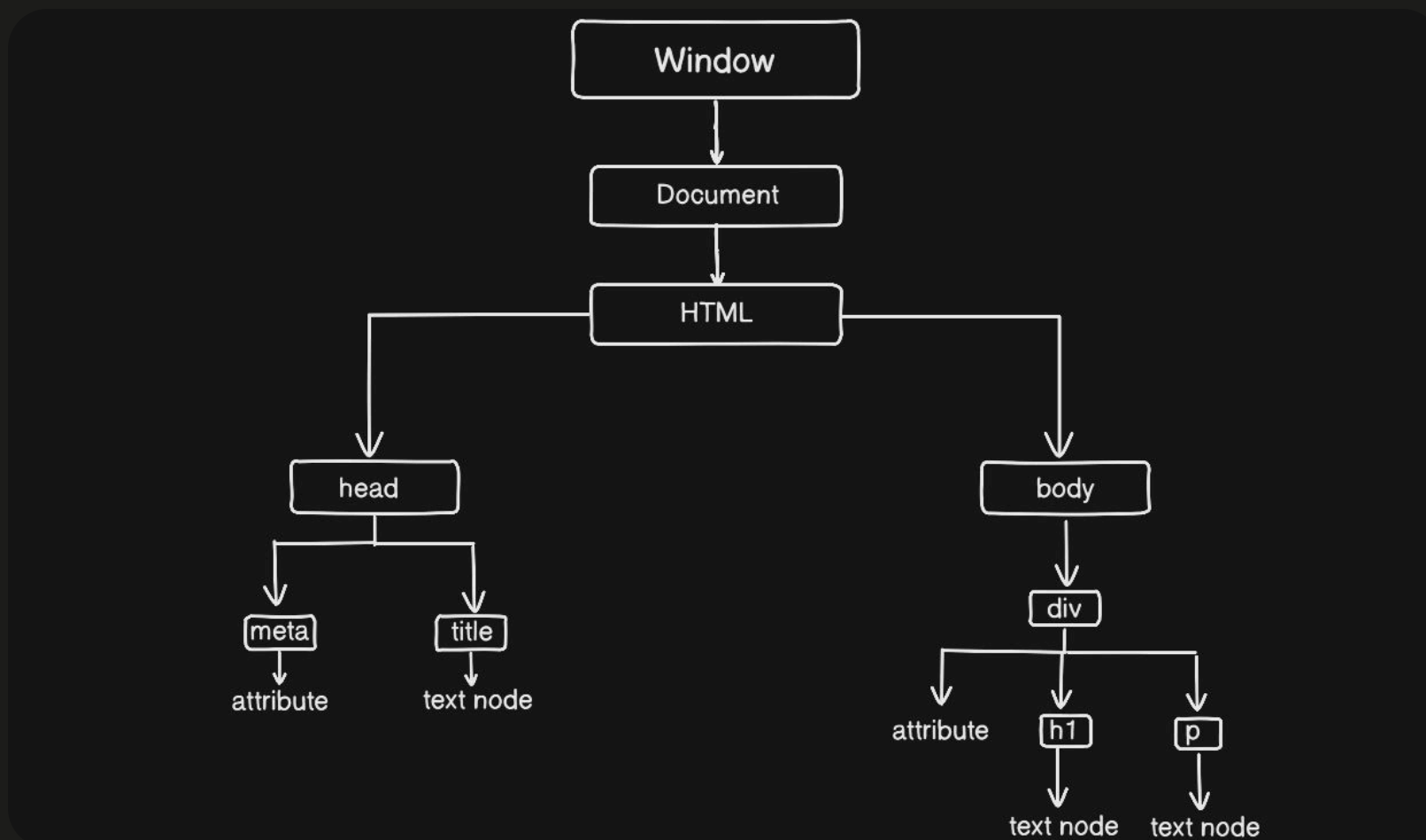
Lec-10

Document Object Model (DOM)

- The HTML Document Object Model (**DOM**) is an **object model for HTML Documents**.
- HTML DOM is a **tree of nodes that represent an HTML page**.
- Suppose we have a following HTML code.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>DOM</title>
  </head>
  <body>
    <div class="bg-black">
      <h1>Intro To DOM</h1>
      <p>Lorem ipsum dolor sit amet.</p>
    </div>
  </body>
</html>
```

above code can be represented in a DOM Tree Structure by following way



- The DOM API provides us with the ability to:
 1. **Find and select elements.**
 2. **Change element content and attributes.**
 3. **Add, remove, or modify elements.**
 4. **Change CSS styles.**
 5. **Add event listeners to react to user input.**
- Is a standard for how to get, change, add, or delete HTML DOM element.
- Using JavaScript we can access the DOM through the API inside our browser.

DOM API Methods and properties

- Developers use global objects like **document** and **window** as entry points to any API.
- If you want to access any element in an HTML page, you always start with accessing the document object. The **document object** represents your **web page**.
- To manipulate HTML with JavaScript, you first need to **select an element**.

Selecting DOM elements

1. getElementById()

- The easiest way to find an HTML element in the DOM, is by **using an element id**.
- This eg find the element with **id="first-head"**.
- **Eg:**

HTML

```
<h1 id="first-head">
  Lorem ipsum dolor sit amet.
</h1>
```

JS

```
console.log(document.getElementById('first-head'))

/* Output:
<h1 id="first-head">Lorem ipsum dolor sit amet.</h1>
*/
```

- If the **element is found**, the **method will return the element as an object** (in element).
- If the **element is not found**, element will contain **null**.

2. getElementsByTagName()

- Returns an **HTML collection** of **all the elements using that specific HTML tag** .
- This eg find the **<p>** elements.
- In order to **iterate over the HTML collection** it **first needs to be converted into an array**.
- **Eg:**

HTML

```
<p>Lorem ipsum dolor sit amet.</p>

<p id="main-para">Lorem ipsum dolor sit, amet
consectetur adipisicing elit. Ipsa, ab?</p>

<p>Lorem ipsum dolor sit amet.</p>

<p>Lorem ipsum dolor sit amet.</p>
```

JS

```
console.log(document.getElementsByTagName('p'))

/*
Output:
HTMLCollection(4) [p, p#main-para, p, p, main-para:
p#main-para]
*/
```

- Below eg finds all **<p>** tags inside a **<div>** with an **id="container-1"**.
- **Eg:**

HTML

```
<div id="container-1">

  <h1>Lorem, ipsum dolor.</h1>
  <p>Lorem ipsum dolor sit amet.</p>
  <h1>Lorem ipsum, dolor sit amet consectetur
adipisicing elit.</h1>
  <p>Lorem ipsum dolor sit amet, consectetur
adipisicing elit. Itaque, quos.</p>

</div>
```

JS

```
let containerVar1 =
document.getElementById('container-1')

console.log(containerVar1.getElementsByTagName('p'))

/*
Output:
HTMLCollection(2) [p, p]
*/
```

Selecting DOM Elements

3. getElementByClassName()

- It is used to find all HTML elements with the **same class name**.
- It also **returns the result in the form of HTML collection**.
- In order to **iterate over the HTML collection** it **first needs to be converted into an array**.

HTML

```
<ul>
  <li class="intro">Intro One</li>
  <li class="intro">Intro Two</li>
  <li class="intro">Intro Three</li>
</ul>
```

JS

```
console.log(document.getElementsByClassName('intro')
)

/* Output:
HTMLCollection(3) [li.intro, li.intro, li.intro]
*/
```

4. querySelector()

- Returns the **first element** that matches a specified CSS selector string.
- This works easily with **Element selectors**, **Class selectors**, **Id selectors** and **attribute selectors** of CSS.

HTML

```
<form action="submit">

  <label for="name">Name</label>
  <input type="text" name="" id="" class="input">
  <br>
  <br>
  <label for="email">Email</label>
  <input type="email" name="" id="" class="input">
  <br>
  <br>
  <button class="btn">Submit</button>

</form>
```

JS

```
console.log(document.querySelector('.input'))

/*Output:
<input type="text" name="" id="" class="input">
*/
```

5. querySelectorAll()

- Returns a **static NodeList** containing **all elements** matching the given CSS selector.
- This works easily with **Element selectors**, **Class selectors**, **Id selectors** and **attribute selectors** of CSS.
- It supports forEach() loop without converting the nodeList into an array first

HTML

```
<form action="submit">

  <label for="name">Name</label>
  <input type="text" name="" id="" class="input">
  <br>
  <br>
  <label for="email">Email</label>
  <input type="email" name="" id="" class="input">
  <br>
  <br>
  <button class="btn">Submit</button>

</form>
```

JS

```
console.log(document.querySelectorAll('.input'))

/* Output:
NodeList(2) [input.input, input.input]
*/
```

Properties to Change HTML contents

1. document.innerHTML

- Returns the entire HTML markup found inside the targeted element.
- It includes all HTML tags, attributes, and text. Writing to this property compiles and renders new HTML elements.

HTML

```
<h1 id="demo">Lorem ipsum dolor sit amet consectetur  
adipisicing elit.  
<a href="#" target="_blank" style="display:  
none;">Link</a> ipsum maiores inventore eveniet!</h1>
```

JS

```
let headingOne = document.getElementById('demo')  
  
console.log(headingOne.innerHTML)  
  
/*  
Output:  
  
'Lorem ipsum dolor sit amet consectetur adipisicing elit.  
<a href="#" target="_blank" style="display:  
none;">Link</a> ipsum maiores inventore eveniet!'  
  
*/
```

2. document.innerText

- Returns only the **visible text content that appears on the browser screen**.
- It respects CSS styling and hides elements with **display: none**.

HTML

```
<h1 id="demo">Lorem ipsum dolor sit amet consectetur  
adipisicing elit.  
<a href="#" target="_blank" style="display:  
none;">Link</a> ipsum maiores inventore eveniet!</h1>
```

JS

```
let headingOne = document.getElementById('demo')  
  
console.log(headingOne.innerText)  
  
/*  
Output:  
  
'Lorem ipsum dolor sit amet consectetur ad'Lorem  
ipsum dolor sit amet consectetur adipisicing elit. ipsum  
maiores inventore eveniet!'  
  
*/
```

3. document.textContent

- Returns the **raw text content of all elements, including hidden text and spacing**.
- It ignores styling and does not trigger a layout calculation. It is highly secure against script injection attacks

HTML

```
<h1 id="demo">Lorem ipsum dolor sit amet  
consectetur adipisicing elit.  
<a href="#" target="_blank" style="display:  
none;">Link</a> ipsum maiores inventore eveniet!</h1>
```

JS

```
let headingOne = document.getElementById('demo')  
  
console.log(headingOne.textContent)  
  
/*  
Output:  
  
'Lorem ipsum dolor sit 'Lorem ipsum dolor sit amet  
consectetur adipisicing elit. Link ipsum maiores  
inventore eveniet!'  
  
*/
```


Property to change CSS Style

- We can change style of an HTML element using **.style** property in JavaScript.
- Eg :

HTML

```
<button class="btn">Submit</button>
```

JS

```
submitBtn = document.querySelector('.btn')
```

```
submitBtn.style.backgroundColor = 'black'
```

```
submitBtn.style.color = 'white'
```



Accessing and Changing Attributes of an elements

1. **getAttribute():**

- Retrieves the current value of a specified attribute on an element
- It returns the value as a **string**. If the attribute does not exist, it returns **null**.
- Eg

HTML

```
<h1 id="head" class="imp">Lorem ipsum dolor sit  
amet.</h1>
```

JS

```
let heading1 = document.getElementById('head')  
console.log(heading1);
```

```
console.log(heading1.getAttribute('class'));  
// will print imp
```

2. **setAttribute():**

- Adds a new attribute to an element or updates the value of an existing one.
- It requires two arguments: the name of the attribute and the value you want to assign to it. If the value is not a string, JavaScript automatically converts it to one
- Eg

HTML

```
<h1 id="head" class="imp">Lorem ipsum dolor sit  
amet.</h1>
```

JS

```
let heading1 = document.getElementById('head')  
console.log(heading1);
```

```
heading1.setAttribute('class','new imp')  
console.log(heading1.getAttribute('class'));  
// will print new imp
```